

Dynamic Reconfiguration Guide (global `nginx.*` namespace)

Table of Contents

1. Overview	2
1.1. The nginx global object at a glance	3
1.2. Object hierarchy	4
2. <code>nginx.version</code> and <code>nginx.cpu_count</code>	5
3. <code>nginx.log(level, message)</code>	5
3.1. Signature	5
4. <code>nginx.cycle</code> (<code>NginxCycle</code>)	6
4.1. Properties	6
4.2. Reading cycle properties	7
4.3. Setting workers count	7
4.4. <code>nginx.cycle.sockets[]</code>	8
5. <code>nginx.events</code> (<code>NginxEvents</code>)	8
5.1. Properties	8
5.2. Tuning accept behaviour at startup	9
6. <code>nginx.stream</code>	9
6.1. Properties	9
6.2. <code>NginxStreamServer</code> properties	10
6.3. <code>NginxStreamProxy</code> sub-object	10
6.4. <code>NginxStreamListener</code>	11
6.5. Stream provisioning example	11
7. <code>nginx.broadcast(fn)</code>	11
7.1. Signature	11
7.2. Behaviour	12
7.3. Use cases	12
8. <code>nginx.setTimeout(ms)</code>	13
8.1. Signature	13
8.2. Behaviour	13
8.3. Worker-only restriction	14
8.4. Example: polling loop with back-off	14
9. <code>nginx.suspendAcceptance()</code> and <code>nginx.resumeAcceptance()</code>	14
9.1. Signatures	15
9.2. Behaviour	15
9.3. Use case: atomic multi-field mutation	15
10. <code>nginx.suspendAllWorkers()</code> and <code>nginx.resumeAllWorkers()</code>	16

10.1. Signatures	16
10.2. Behaviour	16
10.3. Timeout	16
10.4. <code>nginx.withSuspendedAcceptance(fn)</code>	17
10.5. Example: cluster-wide atomic server reconfiguration	17
11. <code>nginx.createSocket(address)</code>	18
11.1. Signature	18
11.2. Phase A vs Phase F	18
12. <code>nginx.workerMemoryLimit</code> and <code>nginx.workerRequestTimeout</code>	19
12.1. <code>workerMemoryLimit</code>	19
12.2. <code>workerRequestTimeout</code>	19
13. Complete examples	19
13.1. Example 1: Startup audit — dump the full <code>nginx</code> object tree	19
13.2. Example 2: Adaptive worker count	20
13.3. Example 3: Per-worker warm-up with <code>setTimeout</code>	21
13.4. Example 4: Graceful config reload with cluster-wide suspension	21
13.5. Example 5: Stream server property inspection and tuning	22
13.6. Example 6: Dynamic stream listener provisioning	22
14. API Reference	22
14.1. <code>nginx</code> (global object)	22
14.2. <code>NginxCycle</code>	23
14.3. <code>NginxEvents</code>	24
14.4. <code>NginxStreamServer</code>	24
14.5. <code>NginxStreamProxy</code>	25
14.6. <code>NginxStreamListener</code>	25
15. Design notes	25
15.1. Why <code>nginx.cycle.workers</code> is writable	25
15.2. <code>broadcast()</code> and the <code>init_process</code> hook	25
15.3. <code>withSuspendedAcceptance</code> is pure JS	26
15.4. <code>workerRequestTimeout</code> and QuickJS interrupts	26



DRAFT FOR COMMENTS — This document is a work in progress.

1. Overview

This guide is a comprehensive reference for the top-level `nginx` global object that is available in every JavaScript context created by the JS module. It covers all properties and methods directly on `nginx.*` — from read-only version information to cross-process coordination primitives — plus the two sub-namespaces `nginx.cycle` and `nginx.events` that expose core and event-loop configuration.

Related guides for the deeper sub-namespaces:

- [Guide 6 \(HTTP\)](#) — `nginx.http.*`
- [Guide 5 \(Servers\)](#) — HTTP server API
- [Guide 4 \(Locations\)](#) — HTTP location API
- [Guide 2 \(All Workers\)](#) — cross-worker coordination
- [Guide 3 \(Checkpoint/Restore\)](#) — snapshot API

1.1. The nginx global object at a glance

```
// Read-only metadata
nginx.version           // "1.27.x" – nginx version string
nginx.cpu_count         // number of logical CPUs

// Logging
nginx.log(level, message) // write to error log

// Sub-namespaces
nginx.cycle             // NginxCycle – master-process config
nginx.events            // NginxEvents – events{} block
nginx.http              // NginxHttp  – HTTP subsystem
nginx.stream            // plain object – stream{} subsystem

// Socket provisioning
nginx.createSocket('host:port') // → NginxSocket (Phase A pre-fork / Phase F post-fork)

// Worker bootstrap
nginx.broadcast(fn)       // register per-worker startup callback

// Async timer
nginx.setTimeout(ms)      // → Promise<undefined>

// Acceptance control – single worker (Phase 1)
nginx.suspendAcceptance() // disable accept events in this worker
nginx.resumeAcceptance()  // re-enable accept events in this worker

// Acceptance control – all workers (Phase 2)
nginx.suspendAllWorkers() // → Promise<undefined>
nginx.resumeAllWorkers()  // → Promise<undefined>
nginx.withSuspendedAcceptance(fn) // convenience wrapper – suspend → fn() → resume

// Per-worker resource limits (written in config phase)
nginx.workerMemoryLimit   // r/w number – JS heap cap per worker (bytes, 0 = none)
nginx.workerRequestTimeout // r/w number – per-request JS timeout (ms, 0 = none)
```

1.2. Object hierarchy

```
nginx
├── version                (string, r/o)
├── cpu_count              (number, r/o)
├── log(level, msg)
├── workerMemoryLimit      (number, r/w)
├── workerRequestTimeout   (number, r/w)
├── cycle                  NginxCycle
│   ├── hostname           (r/o)
│   ├── prefix             (r/o) – config directory
│   ├── installPrefix      (r/o) – installation directory
│   ├── confFile           (r/o)
│   ├── errorLog           (r/o)
│   ├── pid                (r/o) – pid file path
│   ├── workers            (r/w) – worker_processes count
│   ├── connectionN       (r/o) – max connections per worker
│   ├── daemon             (r/o boolean)
│   ├── master             (r/o boolean) – master_process on/off
│   ├── timerResolution    (r/o ms)
│   ├── shutdownTimeout    (r/o ms)
│   ├── priority           (r/o) – worker_priority nice value
│   ├── rlimitNofile       (r/o) – worker_rlimit_nofile
│   ├── workingDirectory   (r/o)
│   └── sockets[]          (r/o) – all listening sockets
├── events                 NginxEvents
│   ├── connections        (r/o) – worker_connections
│   ├── use                 (r/o) – "epoll" / "kqueue" / ...
│   ├── multiAccept        (r/w boolean)
│   ├── acceptMutex        (r/w boolean)
│   └── acceptMutexDelay    (r/w ms)
├── http                   NginxHttp – see Guide 6
│   ├── servers[]
│   ├── sockets[]
│   ├── variables{}
│   ├── upstreams[]
│   └── ...
├── stream                 plain object
│   ├── servers[]          NginxStreamServer[]
│   ├── upstreams[]        NginxStreamUpstream[]
│   ├── sockets[]
│   ├── serverNamesHashMaxSize / BucketSize
│   ├── variablesHashMaxSize / BucketSize
│   └── attach(sock)       → NginxStreamListener
└── createSocket(addr)     → NginxSocket
```

```
└── broadcast(fn)
└── setTimeout(ms)      → Promise
└── suspendAcceptance()
└── resumeAcceptance()
└── suspendAllWorkers() → Promise
└── resumeAllWorkers()  → Promise
└── withSuspendedAcceptance(fn) → Promise
```

2. nginx.version and nginx.cpu_count

```
nginx.version:  string (read-only)
nginx.cpu_count: number (read-only)
```

nginx.version — the nginx version string as compiled in, e.g. "1.27.3".

nginx.cpu_count — the number of logical CPUs detected by nginx at startup (the value of `ngx_ncpu`). Useful for setting **nginx.cycle.workers** to an appropriate value.

```
nginx.log(4, `nginx ${nginx.version} on ${nginx.cpu_count} CPUs`);

// Ensure workers = CPU count at startup
nginx.cycle.workers = nginx.cpu_count;
```

3. nginx.log(level, message)

Writes a message to the nginx error log at the given log level.

3.1. Signature

```
nginx.log(level: number, message: string): undefined
```

level — numeric log level:

Value	Level
1	<code>stderr</code> (always printed)
2	<code>emerg</code>
3	<code>alert</code>
4	<code>crit</code>
5	<code>error</code>
6	<code>warn</code>

Value	Level
7	notice
8	info
9	debug

`message` — the string to write. The prefix `"js: "` is prepended automatically.

```
nginx.log(6, 'upstream weights updated');
nginx.log(8, `servers: ${nginx.http.servers.length}`);
```



`nginx.log` is available in all contexts: config phase (`js_source`), worker request handlers, SharedWorker threads, and JS Worker threads.

4. `nginx.cycle` (NginxCycle)

The `nginx.cycle` object exposes the `ngx_cycle_t` structure — the central nginx process context. It is available in all JS contexts.

4.1. Properties

Property	Type	R/W	Description
<code>hostname</code>	string	R	Machine hostname.
<code>prefix</code>	string	R	Configuration directory (<code>--prefix</code> at configure time, or the directory containing the config file). Equivalent to <code>conf_prefix</code> in nginx source.
<code>installPrefix</code>	string	R	Installation directory (<code>cycle→prefix</code>). Typically <code>/usr/local/nginx</code> or the value passed to <code>--prefix</code> at build time.
<code>confFile</code>	string	R	Full path to the nginx configuration file.
<code>errorLog</code>	string	R	Path to the error log file.
<code>pid</code>	string	R	Path to the nginx pid file.
<code>workers</code>	number	R/W	Current <code>worker_processes</code> count. Write this during config phase to change how many worker processes nginx should fork.
<code>connectionN</code>	number	R	Maximum number of connections per worker (<code>worker_connections</code> × internal allocation).
<code>daemon</code>	boolean	R	Whether nginx runs in daemon mode.
<code>master</code>	boolean	R	Whether the <code>master_process</code> directive is on.

Property	Type	R/W	Description
<code>timerResolution</code>	number (ms)	R	<code>timer_resolution</code> directive value (ms). Zero means disabled.
<code>shutdownTimeout</code>	number (ms)	R	<code>worker_shutdown_timeout</code> value (ms). Zero means disabled.
<code>priority</code>	number	R	<code>worker_priority</code> nice value (0 = default).
<code>rlimitNofile</code>	number	R	<code>worker_rlimit_nofile</code> value. -1 if not set.
<code>workingDirectory</code>	string	R	<code>working_directory</code> directive value. Empty string if not set.
<code>sockets[]</code>	array	R	Snapshot of all listening sockets across HTTP and stream subsystems. Same format as <code>nginx.http.sockets[]</code> and <code>nginx.stream.sockets[]</code> combined into a single flat array.

4.2. Reading cycle properties

```
const c = nginx.cycle;
nginx.log(8, [
  `hostname:      ${c.hostname}`,
  `prefix:       ${c.prefix}`,
  `installPrefix: ${c.installPrefix}`,
  `confFile:     ${c.confFile}`,
  `errorLog:     ${c.errorLog}`,
  `pid:         ${c.pid}`,
  `workers:      ${c.workers}`,
  `connectionN:  ${c.connectionN}`,
  `daemon:       ${c.daemon}`,
  `master:       ${c.master}`,
  `timerResolution:${c.timerResolution} ms`,
  `shutdownTimeout:${c.shutdownTimeout} ms`,
  `priority:     ${c.priority}`,
  `rlimitNofile:  ${c.rlimitNofile}`,
  `workingDir:   ${c.workingDirectory}`,
  `sockets:      ${c.sockets.length}`,
].join('\n'));
```

4.3. Setting workers count

`nginx.cycle.workers` is the only writable cycle property. Write it during config phase (in a `js_source` script) to adjust the worker count before nginx forks. This is equivalent to the `worker_processes` directive.

```
// Auto-size to CPU count (mirrors worker_processes auto;)
```

```
nginx.cycle.workers = nginx.cpu_count;
nginx.log(7, 'workers set to ${nginx.cycle.workers}');
```



Writing `workers` after the fork has no effect on the running process count. Use it only in config-phase scripts.

4.4. `nginx.cycle.sockets[]`

A read-only snapshot array listing every listening socket currently active on this cycle. It combines HTTP and stream sockets. Each entry has:

Property	Type	Description
<code>address</code>	string	Bind address.
<code>port</code>	number	Bind port.
<code>ssl</code>	boolean	Whether TLS is active.
<code>http2</code>	boolean	Whether HTTP/2 is negotiated (HTTP listeners only).
<code>listen</code>	array	Raw listen-directive parameter objects for this port.

```
for (const s of nginx.cycle.sockets) {
    nginx.log(8, `${s.address}:${s.port}  ssl=${s.ssl}`);
}
```

5. `nginx.events` (NginxEvents)

The `nginx.events` object exposes the `events {}` block configuration.

5.1. Properties

Property	Type	R/W	Description
<code>connections</code>	number	R	<code>worker_connections</code> — maximum simultaneous connections per worker. Read-only because the connection pool is allocated at startup.
<code>use</code>	string	R	Event method name, e.g. <code>"epoll"</code> , <code>"kqueue"</code> , <code>"select"</code> .
<code>multiAccept</code>	boolean	R/W	<code>multi_accept on/off</code> — whether a worker accepts as many connections as possible at once. Default: off.
<code>acceptMutex</code>	boolean	R/W	<code>accept_mutex on/off</code> — whether workers take turns accepting new connections (reduces thundering-herd on old kernels).

Property	Type	R/W	Description
<code>acceptMutexDelay</code>	number (ms)	R/W	<code>accept_mutex_delay</code> — delay before trying to acquire the accept mutex.

```
const ev = nginx.events;
nginx.log(8, `events: use=${ev.use} connections=${ev.connections}`);
nginx.log(8, ` multiAccept=${ev.multiAccept} acceptMutex=${ev.acceptMutex}`);
nginx.log(8, ` acceptMutexDelay=${ev.acceptMutexDelay} ms`);
```

5.2. Tuning accept behaviour at startup

```
// Enable multi_accept for high-connection-rate workloads
nginx.events.multiAccept = true;

// Reduce accept-mutex delay for faster hand-off
nginx.events.acceptMutexDelay = 50; // ms (default 500)
```



`multiAccept` and `acceptMutex` are read by the event loop at worker start. Changing them post-fork has no effect on accepted connections but will be visible to future reads of the property.

6. nginx.stream

The `nginx.stream` plain object mirrors `nginx.http` for the TCP/UDP stream subsystem.

6.1. Properties

Property	Type	Description
<code>servers[]</code>	<code>NginxStreamServer[]</code>	Array of all <code>server {}</code> blocks from the <code>stream {}</code> config.
<code>upstreams[]</code>	<code>NginxStreamUpstream[]</code>	Array of all <code>upstream {}</code> blocks in the stream context.
<code>sockets[]</code>	array	Listening socket descriptors for the stream subsystem.
<code>serverNamesHashMaxSize</code>	number (r/o)	<code>server_names_hash_max_size</code> in the stream context.
<code>serverNamesHashBucketSize</code>	number (r/o)	<code>server_names_hash_bucket_size</code> in the stream context.
<code>variablesHashMaxSize</code>	number (r/o)	<code>variables_hash_max_size</code> in the stream context.
<code>variablesHashBucketSize</code>	number (r/o)	<code>variables_hash_bucket_size</code> in the stream context.

Property	Type	Description
<code>attach(sock)</code>	method	Wires a <code>NginxSocket</code> into the stream pipeline. Returns <code>NginxStreamListener</code> . Pre-fork only.

6.2. NginxStreamServer properties

Property	Type	R/W	Description
<code>serverName</code>	string	R	Primary server name (or empty string for unnamed servers).
<code>tcpNodelay</code>	boolean	R/W	<code>tcp_nodelay</code> on/off.
<code>prereadBufferSize</code>	number	R/W	<code>preread_buffer_size</code> (bytes).
<code>prereadTimeout</code>	number (ms)	R/W	<code>preread_timeout</code> .
<code>resolverTimeout</code>	number (ms)	R/W	<code>resolver_timeout</code> .
<code>proxyProtocolTimeout</code>	number (ms)	R/W	<code>proxy_protocol_timeout</code> .
<code>proxy</code>	<code>NginxStreamProxy</code>	R	Stream proxy sub-object.
<code>access</code>	<code>NginxStreamAccess</code>	R	Access control sub-object.
<code>ssl</code>	<code>NginxStreamSSL</code>	R	SSL/TLS sub-object.
<code>handler</code>	function	W	Request handler: <code>function(session)</code> .

6.3. NginxStreamProxy sub-object

Property	Type	R/W	Description
<code>connectTimeout</code>	number (ms)	R/W	<code>proxy_connect_timeout</code> .
<code>timeout</code>	number (ms)	R/W	<code>proxy_timeout</code> .
<code>nextUpstreamTimeout</code>	number (ms)	R/W	<code>proxy_next_upstream_timeout</code> .
<code>bufferSize</code>	number (bytes)	R/W	<code>proxy_buffer_size</code> .
<code>requests</code>	number	R	<code>proxy_requests</code> .
<code>responses</code>	number	R	<code>proxy_responses</code> .
<code>nextUpstreamTries</code>	number	R/W	<code>proxy_next_upstream_tries</code> .
<code>nextUpstream</code>	boolean	R/W	<code>proxy_next_upstream</code> on/off.
<code>proxyProtocol</code>	boolean	R/W	<code>proxy_protocol</code> on/off.
<code>halfClose</code>	boolean	R/W	<code>proxy_half_close</code> on/off.

Property	Type	R/W	Description
<code>socketKeepalive</code>	boolean	R/W	<code>proxy_socket_keepalive</code> on/off.
<code>pass</code>	string	R	<code>proxy_pass</code> target (upstream name or address).

6.4. NginxStreamListener

Returned by `nginx.stream.attach(sock)`.

Member	R/W	Description
<code>address</code>	R	Bind address string.
<code>socket</code>	R	The <code>NginxSocket</code> this listener was created from.
<code>serverNames</code>	R	Array of registered server-name strings.
<code>serverByName(name)</code>	—	Returns <code>NginxStreamServer</code> or <code>null</code> .
<code>addServer(srv)</code>	—	Phase C — set default stream server, activate listener.
<code>addVirtualServer(srv)</code>	—	Phase D — add named virtual stream server.

6.5. Stream provisioning example

```
// Provision a new stream listener at config phase

const sock = nginx.createSocket('0.0.0.0:5555');
const listener = nginx.stream.attach(sock);

// Use an existing stream server as the default
const defaultSrv = nginx.stream.servers[0];
listener.addServer(defaultSrv);

nginx.log(7, 'stream port 5555 ready');
```

7. nginx.broadcast(fn)

Registers a function to be called in every nginx worker process.

7.1. Signature

```
nginx.broadcast(fn: function): undefined
```

7.2. Behaviour

The behaviour differs depending on **where** `broadcast` is called:

Context	Behaviour
Config phase (<code>js_source</code> top-level code, master process)	<code>fn</code> is appended to the internal <code>nginx.__broadcast_queue</code> . After the master forks, each worker calls every queued function during <code>init_process</code> , with the worker context already set. This lets you register per-worker startup logic that requires worker-only APIs (e.g. <code>nginx.setTimeout</code> , request-level COM objects).
Worker request handler (post-fork)	<code>fn</code> is called immediately in the current worker only. No IPC to other workers is performed.

Config phase (master)

Workers (after fork)

```
nginx.broadcast(fn)
  |
  |   appended to
  |   [ __broadcast_queue
  |     [ fn, fn2, ... ]
```

```
init_process:
  calls fn() in Worker 0
  calls fn() in Worker 1
  calls fn() in Worker 2 ...
```

7.3. Use cases

Registering per-worker state — set up a cache, open a file descriptor, or start a background loop that needs to run in every worker:

```
// Config phase – register once, runs in every worker at startup

nginx.broadcast(function workerInit() {
  // This runs in every worker after fork
  nginx.log(8, 'worker started, setting up local state');

  // Example: start a periodic health-check timer
  async function healthLoop() {
    while (true) {
      await nginx.setTimeout(5000);
      nginx.log(8, 'worker health tick');
    }
  }
  healthLoop();
});
```

Installing a handler that requires worker context:

```
// Some handlers need the worker context (e.g. nginx.setTimeout)
// nginx.broadcast defers registration until worker init

nginx.broadcast(function() {
  const loc = nginx.http.servers[0].locations.find(l => l.path === '/slow');
  if (loc) {
    loc.handler = async function(req) {
      await nginx.setTimeout(200);
      req.respond(200, {}, 'done\n');
    };
  }
});
```



Functions queued via `broadcast()` in config phase share the same JS runtime and context as the main `js_source` code. They can close over module-level variables defined in the same script.

8. nginx.setTimeout(ms)

Suspends the current async request handler for `ms` milliseconds using nginx's event-loop timer infrastructure.

8.1. Signature

```
nginx.setTimeout(ms: number): Promise<undefined>
```

8.2. Behaviour

- Creates a native `ngx_event_t` timer in the worker's event loop.
- Returns a `Promise` that resolves when the timer fires.
- While suspended, the worker's event loop continues processing other connections and timers normally.
- After the promise resolves, microtasks are drained and any suspended nginx requests (awaiting the same timer) are finalized.



```
|      | drain microtasks  
|      | ngx_js_async_check → finalize request  
|_____| execution resumes here
```

8.3. Worker-only restriction

`setTimeout` requires a worker context opaque (set after fork). Calling it from config-phase top-level code throws an error. Use `ngx.broadcast` to defer `setTimeout` calls to worker startup:

```
// WRONG – throws at config phase:  
// await ngx.setTimeout(100);  
  
// CORRECT – deferred to worker:  
ngx.broadcast(async function() {  
  await ngx.setTimeout(100);  
  ngx.log(8, 'post-startup delay done');  
});
```

8.4. Example: polling loop with back-off

```
ngx.broadcast(async function() {  
  let delay = 1000;  
  const MAX_DELAY = 30000;  
  
  while (true) {  
    await ngx.setTimeout(delay);  
    try {  
      checkUpstreamHealth();  
      delay = 1000;          // reset on success  
    } catch (e) {  
      ngx.log(5, `health check failed: ${e.message}`);  
      delay = Math.min(delay * 2, MAX_DELAY);  
    }  
  }  
});
```

9. `ngx.suspendAcceptance()` and `ngx.resumeAcceptance()`

Per-worker acceptance control (Phase 1). These are synchronous, instant, and affect only the calling worker.

9.1. Signatures

```
nginx.suspendAcceptance(): undefined
nginx.resumeAcceptance(): undefined
```

9.2. Behaviour

suspendAcceptance() — removes all listening socket read events from this worker's event loop. Incoming TCP connections continue to queue in the OS SYN backlog (up to `net.core.somaxconn`) and will be accepted once **resumeAcceptance()** is called. **Other workers continue accepting normally.**

resumeAcceptance() — re-adds the listening socket read events. No-op if acceptance was not suspended.

Both functions are only callable from a worker process. Calling from config phase throws a `TypeError`.

9.3. Use case: atomic multi-field mutation

Wrap a batch of COM mutations that must appear atomic from new connections' perspective. Without suspension, a new connection could arrive between two mutations and observe a partially-updated configuration:

```
function atomicUpdate(req) {
  nginx.suspendAcceptance();
  try {
    // Both mutations applied before any new connection is accepted
    nginx.http.upstreams[0].peers[0].weight = 200;
    nginx.http.servers[0].locations
      .find(l => l.path === '/api').root = '/var/www/api-v2';
  } finally {
    nginx.resumeAcceptance();
  }
  req.respond(200, {}, 'updated\n');
}
```

```
Worker N
├─ nginx.suspendAcceptance()  ← remove listen read events
│   (new conns queue in OS backlog)
│
├─ mutation 1 (upstream weight)
├─ mutation 2 (location root)
│
└─ nginx.resumeAcceptance()   ← restore listen read events
    (queued conns now accepted with updated config)
```



Because this is per-worker, parallel workers may accept connections with the old config while this worker is suspended. For cluster-wide atomicity use `suspendAllWorkers()` instead.

10. `nginx.suspendAllWorkers()` and `nginx.resumeAllWorkers()`

Cross-worker acceptance control (Phase 2). Asynchronous; coordinates through the manager process.

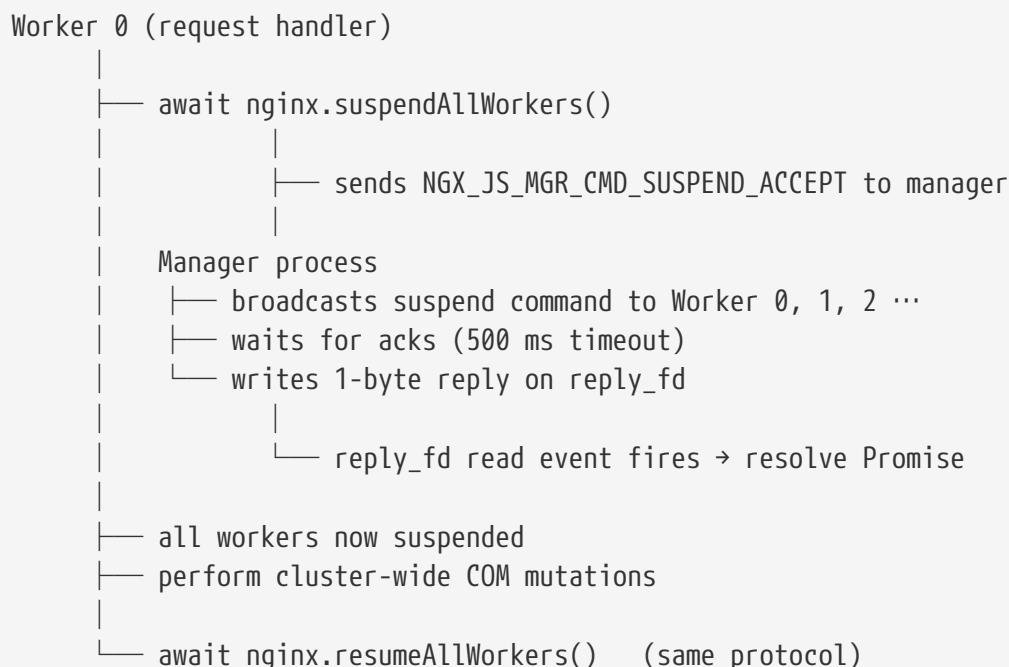
10.1. Signatures

```
nginx.suspendAllWorkers(): Promise<undefined>
nginx.resumeAllWorkers(): Promise<undefined>
```

10.2. Behaviour

Both functions return a `Promise` that resolves after **every** worker in the `nginx` pool has acknowledged the suspend/resume command.

Protocol:



10.3. Timeout

If not all workers ack within 500 ms the manager resolves anyway so the calling worker is not stuck. Slow workers will resume acceptance on their own when the next `resumeAllWorkers`

broadcast arrives.

10.4. `nginx.withSuspendedAcceptance(fn)`

A convenience wrapper defined in pure JavaScript during `ngx_js_com_init`:

```
nginx.withSuspendedAcceptance = async function(fn) {
  await nginx.suspendAllWorkers();
  try {
    await fn();
  } finally {
    await nginx.resumeAllWorkers();
  }
};
```

Use this instead of the raw pair to guarantee `resumeAllWorkers` is called even if `fn` throws.

10.5. Example: cluster-wide atomic server reconfiguration

```
async function hotReloadServers(req) {
  await nginx.withSuspendedAcceptance(async () => {
    // All workers suspended; no new connections accepted anywhere

    // Remove old API server
    nginx.http.removeServer('api-v1.example.com');

    // Add new API server
    const srv = nginx.http.addServer('api-v2.example.com');
    srv.addLocation('/').handler = apiV2Handler;

    // Rebuild routing hash
    nginx.http.rebuildVhostDispatch();
  });
  // All workers resumed with new config

  req.respond(200, {}, 'hot reload complete\n');
}
```

Worker 0 (request handler)

```

|
|
| — await nginx.withSuspendedAcceptance(async () => {
|
|     suspendAllWorkers()
|       → manager broadcasts suspend → all workers ack
|
|
|

```

```

    |
    | └─ removeServer('api-v1.example.com')
    | └─ addServer('api-v2.example.com') + addLocation
    | └─ rebuildVhostDispatch()
    |
    |   resumeAllWorkers()
    |   → manager broadcasts resume → all workers ack
    |
    └─ Promise resolves
    |
    └─ req.respond(200, {}, 'done\n')

```



Worker 0 is also suspended during this window (it cannot accept new connections). In-flight requests on Worker 0 continue executing.

11. nginx.createSocket(address)

Creates a TCP listening socket. See [Guide 6](#) for the full NginxSocket API and the Phase A-D provisioning pipeline.

11.1. Signature

```
nginx.createSocket(address: string): NginxSocket
```

11.2. Phase A vs Phase F

Phase	Context	Behaviour
A (pre-fork)	Config phase (<code>js_source</code> top-level code)	Calls <code>socket()</code> + <code>bind()</code> + <code>listen()</code> directly in the master. All workers inherit the fd on fork.
F (post-fork)	Worker request handler	Sends the bind address to the manager thread via <code>AF_UNIX</code> + <code>SCM_RIGHTS</code> . The manager creates the socket and returns the fd to the requesting worker. Use <code>sock.broadcast()</code> to share it with sibling workers.

```

// Config phase (Phase A)
const httpSock  = nginx.createSocket('0.0.0.0:8080');
const httpListen = nginx.http.attach(httpSock);

const streamSock  = nginx.createSocket('0.0.0.0:5000');
const streamListen = nginx.stream.attach(streamSock);

```

12. nginx.workerMemoryLimit and nginx.workerRequestTimeout

Two writable properties that configure per-worker resource limits. They are read by `init_process` (after fork) and cannot be changed at runtime.

12.1. workerMemoryLimit

```
nginx.workerMemoryLimit: number (bytes, 0 = no limit)
```

Caps the QuickJS heap size for each worker's JS runtime. After fork, `init_process` calls `JS_SetMemoryLimit(rt, workerMemoryLimit)` if the value is non-zero.

```
// Limit each worker to 128 MB of JS heap
nginx.workerMemoryLimit = 128 * 1024 * 1024;
```

12.2. workerRequestTimeout

```
nginx.workerRequestTimeout: number (ms, 0 = no timeout)
```

Caps the wall-clock time a single JS request handler invocation may run. After fork, `init_process` installs a QuickJS interrupt handler that checks a monotonic-clock deadline set around each `JS_Call` in the content handler. If the deadline passes, the interrupt handler throws a `SyntaxError` to terminate the runaway script.

```
// Kill any request handler that runs for more than 5 seconds
nginx.workerRequestTimeout = 5000;
```



The timeout applies to synchronous execution time within a single `JS_Call` invocation. Async handlers that `await nginx.setTimeout()` are not penalised for the wall-clock time spent waiting; the deadline is reset at each resume.

13. Complete examples

13.1. Example 1: Startup audit — dump the full nginx object tree

```
(function auditAll() {
  const c = nginx.cycle;
```

```

const ev = nginx.events;

nginx.log(8, '=== nginx global audit ===');
nginx.log(8, `version:    ${nginx.version}`);
nginx.log(8, `cpu_count:  ${nginx.cpu_count}`);

nginx.log(8, `cycle.hostname:    ${c.hostname}`);
nginx.log(8, `cycle.prefix:      ${c.prefix}`);
nginx.log(8, `cycle.installPrefix:  ${c.installPrefix}`);
nginx.log(8, `cycle.confFile:      ${c.confFile}`);
nginx.log(8, `cycle.errorLog:      ${c.errorLog}`);
nginx.log(8, `cycle.pid:          ${c.pid}`);
nginx.log(8, `cycle.workers:        ${c.workers}`);
nginx.log(8, `cycle.connectionN:    ${c.connectionN}`);
nginx.log(8, `cycle.daemon:          ${c.daemon}`);
nginx.log(8, `cycle.master:          ${c.master}`);
nginx.log(8, `cycle.timerResolution:${c.timerResolution} ms`);
nginx.log(8, `cycle.shutdownTimeout:${c.shutdownTimeout} ms`);
nginx.log(8, `cycle.priority:        ${c.priority}`);
nginx.log(8, `cycle.rlimitNofile:    ${c.rlimitNofile}`);
nginx.log(8, `cycle.workingDir:      ${c.workingDirectory}`);

nginx.log(8, `events.use:            ${ev.use}`);
nginx.log(8, `events.connections:      ${ev.connections}`);
nginx.log(8, `events.multiAccept:      ${ev.multiAccept}`);
nginx.log(8, `events.acceptMutex:      ${ev.acceptMutex}`);
nginx.log(8, `events.acceptMutexDelay: ${ev.acceptMutexDelay} ms`);

nginx.log(8, `http.servers:    ${nginx.http.servers.length}`);
nginx.log(8, `http.sockets:    ${nginx.http.sockets.length}`);
nginx.log(8, `stream.servers:  ${((nginx.stream.servers||[]).length)}`);

const allSocks = c.sockets;
nginx.log(8, `total sockets: ${allSocks.length}`);
for (const s of allSocks) {
    nginx.log(8, `  ${s.address}:${s.port} ssl=${s.ssl} http2=${s.http2||false}`);
}
})();

```

13.2. Example 2: Adaptive worker count

Set **workers** to the CPU count minus one (leave a core for the OS) and cap the memory per worker:

```

const cpus = nginx.cpu_count;
nginx.cycle.workers = Math.max(1, cpus - 1);
nginx.workerMemoryLimit = 64 * 1024 * 1024; // 64 MB per worker
nginx.log(7, `workers=${nginx.cycle.workers} memLimit=64MB`);

```

13.3. Example 3: Per-worker warm-up with setTimeout

After fork, each worker runs a warm-up routine that pre-builds caches and logs its startup sequence:

```
nginx.broadcast(async function workerWarmUp() {
  nginx.log(8, 'worker warm-up started');

  // Stagger workers to avoid thundering-herd on a shared resource
  const jitter = Math.floor(Math.random() * 500);
  await nginx.setTimeout(jitter);

  // ... pre-populate caches, connect to local resources ...

  nginx.log(8, 'worker warm-up complete');
});
```

13.4. Example 4: Graceful config reload with cluster-wide suspension

A handler that reloads routing config from an external source and applies it atomically across all workers:

```
async function reloadConfig(req) {
  let newRoutes;
  try {
    // Fetch new routing table (async, workers still accepting)
    newRoutes = await fetchRoutingTable();
  } catch (e) {
    req.respond(500, {}, `fetch failed: ${e.message}\n`);
    return;
  }

  await nginx.withSuspendedAcceptance(async () => {
    // Validate before applying
    for (const { host, path, upstream } of newRoutes) {
      if (!nginx.http.servers.find(s => s.names.includes(host))) {
        nginx.http.addServer(host);
      }
      const srv = nginx.http.servers.find(s => s.names.includes(host));
      const loc = srv.addLocation(path);
      loc.proxyPass = upstream;
    }
    nginx.http.rebuildVhostDispatch();
  });

  req.respond(200, {}, `reload complete (${newRoutes.length} routes)\n`);
}
```

```
}
```

13.5. Example 5: Stream server property inspection and tuning

```
// Inspect and tune all stream servers at startup

for (const srv of nginx.stream.servers) {
  nginx.log(8, `stream server: ${srv.serverName || '<unnamed>'}`);
  if (srv.proxy) {
    nginx.log(8, `  proxy.pass=${srv.proxy.pass}`);
    nginx.log(8, `  proxy.connectTimeout=${srv.proxy.connectTimeout} ms`);

    // Reduce connection timeout for internal servers
    if (srv.proxy.pass && srv.proxy.pass.startsWith('internal_')) {
      srv.proxy.connectTimeout = 1000; // 1 s
      srv.proxy.timeout        = 30000; // 30 s
    }
  }
}
```

13.6. Example 6: Dynamic stream listener provisioning

```
// Provision a new TCP proxy listener at config phase

const sock      = nginx.createSocket('0.0.0.0:7000');
const listener = nginx.stream.attach(sock);

// Use the first existing stream server as default
if (nginx.stream.servers.length > 0) {
  listener.addServer(nginx.stream.servers[0]);
  nginx.log(7, 'stream port 7000 ready');
} else {
  nginx.log(5, 'no stream servers available; skipping port 7000');
}
```

14. API Reference

14.1. nginx (global object)

Member	R/W	Description
<code>version</code>	R	nginx version string (e.g. "1.27.3").

Member	R/W	Description
<code>cpu_count</code>	R	Number of logical CPUs.
<code>log(level, msg)</code>	—	Write to the error log.
<code>cycle</code>	R	<code>NginxCycle</code> — core process configuration.
<code>events</code>	R	<code>NginxEvents</code> — <code>events{}</code> block configuration.
<code>http</code>	R	<code>NginxHttp</code> — HTTP subsystem. See Guide 6.
<code>stream</code>	R	Plain object — <code>stream{}</code> subsystem.
<code>createSocket(addr)</code>	—	Create a TCP listening socket. Returns <code>NginxSocket</code> .
<code>broadcast(fn)</code>	—	Register a per-worker startup callback (or call immediately in worker).
<code>setTimeout(ms)</code>	—	Async timer. Returns <code>Promise<undefined></code> .
<code>suspendAcceptance()</code>	—	Disable accept events in the current worker (synchronous, worker-only).
<code>resumeAcceptance()</code>	—	Re-enable accept events in the current worker (synchronous, worker-only).
<code>suspendAllWorkers()</code>	—	Broadcast suspend to all workers. Returns <code>Promise<undefined></code> .
<code>resumeAllWorkers()</code>	—	Broadcast resume to all workers. Returns <code>Promise<undefined></code> .
<code>withSuspendedAcceptance(fn)</code>	—	Convenience: <code>suspendAllWorkers</code> → <code>fn()</code> → <code>resumeAllWorkers</code> . Returns <code>Promise<undefined></code> .
<code>workerMemoryLimit</code>	R/W	Per-worker JS heap cap in bytes. 0 = no limit. Read at <code>init_process</code> ; config-phase write only.
<code>workerRequestTimeout</code>	R/W	Per-request JS execution timeout in ms. 0 = no timeout. Read at <code>init_process</code> ; config-phase write only.

14.2. NginxCycle

Property	R/W	Description
<code>hostname</code>	R	Machine hostname.
<code>prefix</code>	R	Config directory.
<code>installPrefix</code>	R	Installation directory.
<code>confFile</code>	R	Full config file path.
<code>errorLog</code>	R	Error log path.
<code>pid</code>	R	PID file path.
<code>workers</code>	R/W	Worker process count.

Property	R/W	Description
connectionN	R	Max connections per worker.
daemon	R	Daemon mode (boolean).
master	R	master_process on/off (boolean).
timerResolution	R	timer_resolution (ms).
shutdownTimeout	R	worker_shutdown_timeout (ms).
priority	R	worker_priority nice value.
rlimitNofile	R	worker_rlimit_nofile (-1 if unset).
workingDirectory	R	working_directory ("" if unset).
sockets[]	R	All listening sockets (HTTP + stream).

14.3. NginxEvents

Property	R/W	Description
connections	R	worker_connections.
use	R	Event method name ("epoll", "kqueue", ...).
multiAccept	R/W	multi_accept on/off.
acceptMutex	R/W	accept_mutex on/off.
acceptMutexDelay	R/W	accept_mutex_delay (ms).

14.4. NginxStreamServer

Property	R/W	Description
serverName	R	Primary server name.
tcpNodelay	R/W	tcp_nodelay on/off.
prereadBufferSize	R/W	preread_buffer_size (bytes).
prereadTimeout	R/W	preread_timeout (ms).
resolverTimeout	R/W	resolver_timeout (ms).
proxyProtocolTimeout	R/W	proxy_protocol_timeout (ms).
proxy	R	NginxStreamProxy sub-object.
access	R	NginxStreamAccess sub-object.
ssl	R	NginxStreamSSL sub-object.
handler	W	Session handler function.

14.5. NginxStreamProxy

Property	R/W	Description
<code>connectTimeout</code>	R/W	<code>proxy_connect_timeout</code> (ms).
<code>timeout</code>	R/W	<code>proxy_timeout</code> (ms).
<code>nextUpstreamTimeout</code>	R/W	<code>proxy_next_upstream_timeout</code> (ms).
<code>bufferSize</code>	R/W	<code>proxy_buffer_size</code> (bytes).
<code>requests</code>	R	<code>proxy_requests</code> .
<code>responses</code>	R	<code>proxy_responses</code> .
<code>nextUpstreamTries</code>	R/W	<code>proxy_next_upstream_tries</code> .
<code>nextUpstream</code>	R/W	<code>proxy_next_upstream</code> on/off.
<code>proxyProtocol</code>	R/W	<code>proxy_protocol</code> on/off.
<code>halfClose</code>	R/W	<code>proxy_half_close</code> on/off.
<code>socketKeepalive</code>	R/W	<code>proxy_socket_keepalive</code> on/off.
<code>pass</code>	R	<code>proxy_pass</code> target.

14.6. NginxStreamListener

Member	R/W	Description
<code>address</code>	R	Bind address.
<code>socket</code>	R	Associated NginxSocket.
<code>serverNames</code>	R	Registered server-name strings.
<code>serverByName(name)</code>	—	Returns NginxStreamServer or null.
<code>addServer(srv)</code>	—	Phase C — set default server, activate listener.
<code>addVirtualServer(srv)</code>	—	Phase D — add named virtual server.

15. Design notes

15.1. Why `nginx.cycle.workers` is writable

`workers` is the only writable `NginxCycle` property because it modifies the `worker_processes` field in `ngx_core_conf_t` — a plain integer read at fork time. All other cycle fields are either structural (memory pool layouts, allocated arrays) or already computed at config parse time in ways that cannot be safely changed after-the-fact.

15.2. `broadcast()` and the `init_process` hook

JavaScript code evaluated at config phase (inside `js_source`) runs in the master process before any

workers fork. Functions in `__broadcast_queue` are stored as JS values in the shared context. Because workers inherit the JS runtime via copy-on-write after fork, the queue is accessible in all workers. `init_process` iterates the queue and calls each function with the worker context opaque already installed, so `nginx.setTimeout` and request-level COM objects work correctly inside broadcast callbacks.

15.3. `withSuspendedAcceptance` is pure JS

`withSuspendedAcceptance` is not a C function — it is a short async function defined in a script string evaluated during `ngx_js_com_init`. This makes it easy to inspect or override: `nginx.withSuspendedAcceptance` can be reassigned from JS code for testing purposes.

15.4. `workerRequestTimeout` and QuickJS interrupts

The timeout is implemented with `JS_SetInterruptHandler`. Before each `JS_Call` into a request handler, the C layer records `clock_gettime(CLOCK_MONOTONIC) + workerRequestTimeout` as a deadline. The interrupt handler fires periodically (QuickJS checks at branch-back points) and compares the current time to the deadline. If exceeded it returns 1, which causes QuickJS to throw a `SyntaxError("Script interrupted")`. Async suspend points (`await`) do not advance the deadline; the deadline is re-stamped each time the event loop resumes the handler from a timer or I/O event.